Project Documentation: "Intellectus" - An AI Research Assistant Agent

---

1. Title Page

- **Course Number and Name:** ITAI 2376, Deep Learning Artificial Intelligence
- **Project Title:** "Intellectus" - An AI Research Assistant Agent
- **Group Name:** AI Alchemists
- **Group Members:** Ruben Valenzuela, Benjamin LaCount II
- **Submission Date:** November 17, 2025

2. Problem Statement

- **Problem Description:** Students, researchers, and professionals are overwhelmed by the sheer volume of digital information. The process of finding relevant sources, evaluating their credibility, synthesizing key findings, and properly formatting citations is manually intensive, time-consuming, and prone to error.
- **Problem Importance:** Inefficient research wastes valuable time that could be spent on analysis and original thought. Furthermore, poor source evaluation can lead to weak arguments, misinformation, and academic or professional integrity issues.
- **Beneficiaries:**
  - **Students:** Can quickly gather and understand sources for essays, theses, and reports.
  - **Academics & Researchers:** Can accelerate literature reviews and stay updated on new publications.
  - **Journalists & Analysts:** Can perform rapid background research and fact-checking on complex topics.

3. Project Option

- **Chosen Option:** Research Assistant Agent
- **Justification:** This project provides a practical and high-impact use case for demonstrating advanced agent design.
- **Specific Focus:** Given the ~3.5 week timeline, our focus will be on a Minimum Viable Product (MVP) that excels at the core loop:
    1. **Search:** Execute a web search for a user's topic.
    2. **Read & Summarize:** "Read" the top N results and generate summaries.
    3. **Report & Cite:** Present a consolidated report with structured summaries and basic APA/MLA-formatted citations.
    4. **Feedback:** Implement a simple user feedback mechanism (e.g., "Was this report helpful?") to adjust agent preferences.

4. Agent Design

- **High-Level Architecture Diagram (Text Flow):**
  `User Query -> [Agent Core]`
  `[Agent Core]`
  `1. Input Processor (Validation & Intent Parsing)`
  `2. Reasoning Component (ReAct Pattern)`
  `-> Thought: "I need to find sources for topic X."`
  `-> Action: call_tool(WebSearch, "topic X")`
  `-> Observation: [Tool Output: List of URLs]`
  `-> Thought: "I will read the top 3 URLs."`
  `-> Action: call_tool(DocumentReader, [url1, url2, url3])`
  `-> Observation: [Tool Output: Text content]`
  `-> Thought: "I will summarize this content and format citations."`
  `-> Action: call_tool(Summarizer, "...")`
  `-> Observation: [Tool Output: Final summary]`
  `3. Memory System (Short-term context)`
  `4. Output Generation (Format to Markdown)`
  `-> [Markdown Report]`
- **Main Components:**
  - **Input Processing:** A simple parser that validates the user's query against safety policies (Input Validation) and extracts the core research topic and any parameters (e.g., `topic: "quantum computing", style: "APA"`).
  - **Memory System:** A short-term "scratchpad" (conversation history) that stores the intermediate steps (thoughts, actions, observations) of the ReAct loop.
  - **Reasoning Component:** The core decision-maker, built using the **ReAct (Reasoning and Acting)** pattern. This allows the agent to iteratively build a plan, execute tools, observe the results, and refine its plan until the final report is generated.
  - **Output Generation:** A template-based component that formats the final list of summaries, insights, and citations into a clean Markdown report for the user.
- **Agent Pattern:** We will implement the **ReAct** pattern. This is ideal for a research agent because the task is not a single-step problem. Research requires a dynamic chain of actions: searching, evaluating results, reading, synthesizing, and then potentially searching again based on new findings. ReAct's `Thought-Action-Observation` loop perfectly mirrors this exploratory process.

5. Tool Selection (Open-Source)

- **Tool 1: Web Search (SearxNG)**
  - **Why:** This is the primary tool for information retrieval. **SearxNG** is a free, open-source, and self-hostable metasearch engine that aggregates results from various sources. This gives us control over our information sources and avoids API costs and rate limits from a single provider.
  - **Integration:** We will host our own instance of SearxNG (e.g., in a Docker container). The ReAct loop's `Reasoning Component` will formulate a search query and make a request to our local SearxNG API. The agent will handle errors (e.g., instance down, no results) by logging the error and reporting failure to the user.
- **Tool 2: Document Processing & Retrieval (BeautifulSoup + ChromaDB)**
  - **Why:** A search (Tool 1) only provides URLs. We need a tool to (a) fetch the content from those URLs (**BeautifulSoup**) and (b) store and query that content efficiently (**ChromaDB**) for Retrieval-Augmented Generation (RAG).
  - **Integration:**
    1. **Reader:** After SearxNG returns URLs, a `DocumentReader` sub-tool (using Python's `requests` and `BeautifulSoup`) will scrape the HTML content.
    2. **Vector Store:** This content will be chunked, vectorized (using an open-source model like `Sentence-Transformers`), and indexed in a local **ChromaDB** instance.
    3. **Retrieval:** The agent's `Reasoning Component` will then query this index (e.g., `query_index("What are the main arguments from these sources?")`) to generate grounded summaries, ensuring the agent is "citing" its findings directly from the retrieved documents.

6. Development Plan

- **Methodology:** Agile (Scrum-lite) with two 1.5-week sprints, concluding with a final half-week for integration and deployment.

Timeline	Key Milestones & Tasks	Owner
Sprint 1 (Week 1-2)		
Nov 17 - Nov 21	• Project Setup (Repo, Docker Compose) • Basic Agent Architecture (ReAct loop) • **Tool 1 Integration:** Deploy SearxNG & build API connector	Team A
Nov 24 - Nov 28	• **Tool 2 Integration:** Document Reader (BeautifulSoup) • **Tool 2 Integration:** ChromaDB (Vector Store) • Basic RAG pipeline (Read -> Index -> Query)	Team B
Sprint 2 (Week 2-3)		
Dec 1 - Dec 5	• Output Generation (Report formatting) • **RL Element:** User feedback mechanism (UI button + backend logic) • **Safety Measures:** Input validation & content filtering	Team A
Dec 8 - Dec 12	• **Final Integration & Testing** • **Evaluation:** Run "golden set" of queries • Code freeze, final documentation, and demo prep	All

7. Evaluation Strategy

- **Success Definition:** The agent consistently produces research reports that are **relevant** (address the query), **faithful** (grounded in sources), and **useful** (well-structured and cited).
- **Key Metrics:**
  1. **RAG Triad (Quantitative):**
     - **Context Relevance:** Pct. of retrieved documents that are relevant to the query.
     - **Faithfulness:** Pct. of summary sentences that are directly supported by the source text.
     - **Answer Relevance:** Pct. of the final report that directly answers the user's prompt.
  2. **Citation Accuracy (Quantitative):** Pct. of citations correctly formatted in the requested style (e.g., APA).
  3. **User Feedback Score (Qualitative/RL):** The average score from the user (e.g., 1-5 stars or thumbs up/down) on report quality. This score will be our primary reward signal.
- **Testing Approach:**
  - **Unit Tests:** For each tool integration (e.g., "does the scraper handle a 404?").
  - **Integration Tests:** For the full ReAct loop.
  - **"Golden Set" Testing:** We will create a test suite of 20 diverse research queries (e.g., simple, complex, ambiguous) and manually evaluate the agent's output against our key metrics.

8. Resource Requirements

- **Computational Resources:**
  - A local machine or VM (e.g., 4-core CPU, 16GB RAM) capable of running the agent, a local LLM (if not using an API), and Docker containers for SearxNG and ChromaDB.
- **Libraries & Frameworks:**
  - **Python:** As the core language.
  - **SearxNG:** For self-hosted web search.
  - **ChromaDB:** For the open-source vector database.
  - **BeautifulSoup / Scrapy:** For web scraping.
  - **Sentence-Transformers:** For creating text embeddings.
  - **FastAPI / Flask:** For hosting the agent as an API (optional).
  - **Local LLM (Optional):** e.g., Ollama running Llama 3 or Mistral for summarization and reasoning to be fully offline.
- **Estimated Cost:**
  - **$0** if run entirely on existing local hardware.
  - **~$30-$80/month** if a cloud VM is required for persistent hosting and more powerful compute. Costs are for compute, not API credits.

9. Risk Assessment

Potential Challenge	Mitigation Strategy	Backup Plan
Tool Failure / Stability	Implement robust error handling (e.g., retries) for local services (SearxNG, ChromaDB). Use Docker health checks to monitor services.	Have a static, small set of documents pre-indexed. The agent will only "research" within this set.
Poor Source Credibility	The agent will state a limitation that it cannot guarantee source credibility. We will add a simple heuristic to prioritize .edu, .gov, or known academic domains.	Block-list known misinformation domains.
Hallucination / Ungrounded Summaries	Strictly enforce the RAG pattern. All summaries *must* be generated by querying the indexed text from the sources, not from the agent's base model knowledge.	Fall back to simpler "extractive summarization" (pulling key sentences) instead of abstractive summarization.
Web Scraping Blocks	Use standard scraping etiquette (set User-Agent, respect robots.txt). Implement rotating proxies or a service like ScrapeAPI if this becomes a major issue.	The agent will only be able to read from URLs that do not employ anti-scraping measures and will report failure on others.
Scope Creep	Adhere strictly to the MVP feature set. Any new ideas (e.g., "graph-based source analysis") will be logged for a "v2" but not implemented.	Drop the RL feedback loop and focus only on the core Search-Read-Report loop if time is short.